American Computer Science League

2022-2023 ● Contest 3: Create a Tree ● Junior Division

PROBLEM: Given a string to create 2 parallel arrays, process the letters in the string from left to right, inserting each letter into the first array in alphabetical order, one letter at a time. During this process, each letter is assigned an integer value in the second array as follows:

1. The first letter has a value of 0.
2. If the new letter is the first or last in the array, its value is one more than the value of the adjacent letter.
3. Otherwise, it is one more than the larger value of the two adjacent values. If the letter is already in the array, the new letter is placed before the existing letter.

Once the two arrays are created, output the letters in order of their value, from left to right, starting with the 0, then all of the 1s, then all of the 2s, etc.

EXAMPLE #1: Here is how the array is built for the string **PYTHONN**:
The letter P illustrates Rule 1; Y and H illustrate Rule 2; the T, O and Ns illustrate Rule 3; and the final N illustrates what happens if there is a duplicate letter.

P	*Letters*	P							
	Values	0							
Y	*Letters*	P	Y						
	Values	0	1						
T	*Letters*	P	T	Y					
	Values	0	2	1					
H	*Letters*	H	P	T	Y				
	Values	1	0	2	1				
O	*Letters*	H	O	P	T	Y			
	Values	1	2	0	2	1			
N	*Letters*	H	N	O	P	T	Y		
	Values	1	3	2	0	2	1		
N	*Letters*	H	N	N	O	P	T	Y	

Values	1	4	3	2	0	2	1		

Once the array is created, the output would be PHYOTNN.

EXAMPLE #2: The string **BINARYSEARCHTREE** results in the following array:

Letters	A	A	B	C	E	E	E	H	I	N	R	R	R	S	T	Y
Values	2	1	0	3	5	4	2	3	1	2	5	4	3	5	6	4

The output for this example would be BAIAENCHRERYERST.

INPUT: Input a string to use in building the two arrays. The string will be no more than 80 characters containing all capital letters.

OUTPUT: Output the letters in order of their value, from left to right, starting with the 0, then all of the 1s, then all of the 2s, etc.

SAMPLE INPUT:

1. PYTHONN
2. BINARYSEARCHTREE
3. CORONAVIRUS
4. FINALSFORACSL
5. HACKERRANKPLATFORM

SAMPLE OUTPUT:

1. PHYOTNN
2. BAIAENCHRERYERST
3. CAOORNRVIUS
4. FAIAFNCLSLORS
5. HAKACKRAERTFNLPMOR

American Computer Science League

2022-2023 ● Contest 3: Create a Tree ● Junior Division

TEST DATA

TEST INPUT:

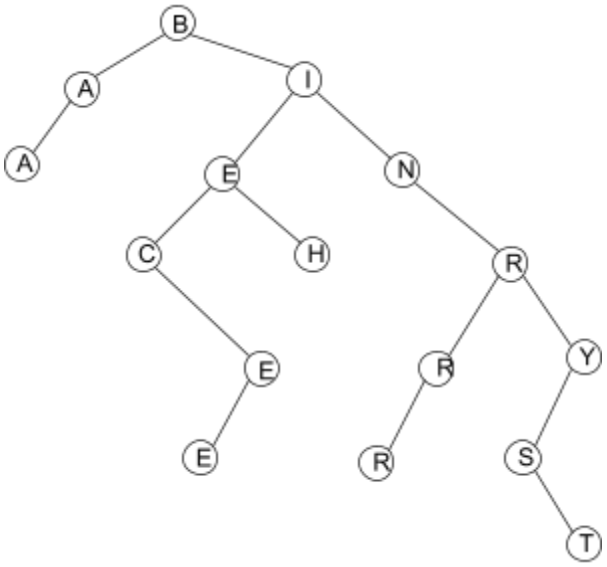
1. MOTHER
2. ACSLCONTEST
3. SUPERCALIFRAGILISTIC
4. JAVAPROGRAMMING
5. ABDFHKMOQTVWYZ

TEST OUTPUT:

1. MHOETR
2. ACCSLTEOTNS
3. SPUERTCLRSAIACFLGIII
4. JAVAGPAGIORMRMN
5. ABDFHKMOQTVWYZ

FOR INFORMATION PURPOSES ONLY

The procedure above is a way to create a binary search tree. The values in the array represent the depth of the nodes. Rearranging the letters above by depth from left to right gives: BAIAENCHRERYERST. The tree below is the result:



American Computer Science League

2022-2023 ● Contest 3: Create a Tree ● Junior Division

问题: 给定一个字符串，用于创建 2 个平行数组，从左到右处理字符串中的字母，按字母表顺序将字母依次插入第一个数组，每次一个字母。在此过程中，在第二个数组中为每个字母分配一个整数值，规则如下所示：

1. 第一个字母的值是 0。
2. 如果新插入的字母是数组中的第一个或最后一个，则它的值比相邻字母的值大 1。
3. 否则，它比两个相邻值中的较大值大 1。如果这个字母已经在数组中，则将新插入的字母放在现有字母之前。

创建两个数组后，按照字母对应整数值的顺序从左到右输出字母，首先输出数值为 0 的字母，然后是所有数值为 1 的字母，然后是所有数值为 2 的字母，以此类推。

示例 #1: 下面展示了如何对字符串 **PYTHONN 创建**数组：

字母 P 表示规则 1；Y 和 H 表示规则 2；T、O 和 N 表示规则 3；最后一个 N 表示如果有重复的字母如何在数组中处理。

P	*字母*	P							
	数值	0							
Y	*字母*	P	Y						
	数值	0	1						
T	*字母*	P	T	Y					
	数值	0	2	1					
H	*字母*	H	P	T	Y				
	数值	1	0	2	1				
O	*字母*	H	O	P	T	Y			
	数值	1	2	0	2	1			
N	*字母*	H	N	O	P	T	Y		
	数值	1	3	2	0	2	1		
N	*字母*	H	N	N	O	P	T	Y	

数值	1	4	3	2	0	2	1		

数组创建好后，输出将会是 PHYOTNN。

示例 #2: 字符串 **BINARYSEARCHTREE** 会得到下列数组：

字母	A	A	B	C	E	E	E	H	I	N	R	R	R	S	T	Y
数值	2	1	0	3	5	4	2	3	1	2	5	4	3	5	6	4

这个示例的输出将会是 BAIAENCHRERYERST。

输入: **输入**一个字符串用于创建 2 个数组，这个字符串中均为大写字母且不会超过 80 个字母。

输出: 按照字母的数值顺序从左到右输出字母，首先输出数值为 0 的字母，然后是所有数值为 1 的字母，然后是所有数值为 2 的字母，以此类推。

样本输入：

1. PYTHONN
2. BINARYSEARCHTREE
3. CORONAVIRUS
4. FINALSFORACSL
5. HACKERRANKPLATFORM

样本输出：

1. PHYOTNN
2. BAIAENCHRERYERST
3. CAOORNRVIUS
4. FAIAFNCLSLORS
5. HAKACKRAERTFNLPMOR

American Computer Science League

2022-2023 ● Contest 3: Create a Tree ● Junior Division

测试数据

测试输入：

1. MOTHER
2. ACSLCONTEST
3. SUPERCALIFRAGILISTIC
4. JAVAPROGRAMMING
5. ABDFHKMOQTVWYZ

测试输出：

1. MHOETR
2. ACCSLTEOTNS
3. SPUERTCLRSAIACFLGIII
4. JAVAGPAGIORMRMN
5. ABDFHKMOQTVWYZ

FOR INFORMATION PURPOSES ONLY

The procedure above is a way to create a binary search tree. The values in the array represent the depth of the nodes. Rearranging the letters above by depth from left to right gives: BAIAENCHRERYERST. The tree below is the result:

